# CSM Week 10

Things in scheme

1) Primitives
   1) numbers
   2) strings → require double quotes
   3) booleans → #t (True) #f (False)
   4) pairs "linked list"
   5) nil "null"

2) Functions
   - (cons ... ...)
   - (car ...)
   - (cdr ...)
   - (list ... ....)
   - (even? ...)
   - (odd? ...)
   - (1 ... ...)
   - (quotient ... ...)
   - (modulo ... ...) same as modulo

(+ 5 4) 5+4

(operator operand)

(cons 1 (cons 2 nil)) Link(1, Link(2))

Link(2, Link. empty)

[1] → [2] → [3]

(car (cons 1 (cons 2 (cons 3 nil)))) → link.first

1

(cdr...) ↔ link.rest

[4] → [3]

(remainder ... ...)

(= ... ...) ==

(eq? ... ...) == for non-pairs and is for pairs

(cdr (cons 1 (cons 2 (cons 3 nil))))

```
scm> (even? 5)
#f
scm> (odd? 5)
#t
scm> (+ 5 3)
8
scm> (1 6 2)
3
scm> (* 6 2)
12
```

Python

5//2 5%2

Scheme

(quotient 5 2)

(modulo 5 2)

(= 5 2)

#f

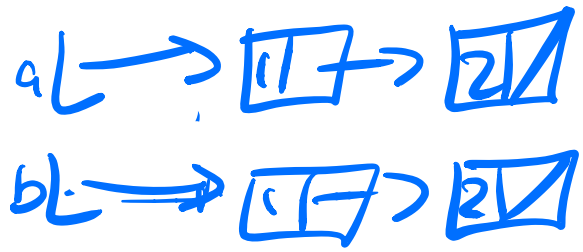
(eq? ... )

(= 2 2)

#t

(= 5 2)

#f




(eq? 2 2)

#t

(eq? 5 2)

#f

(define a (cons 1 (cons 2 nil)))

(define b (cons 1 (cons 2 nil)))

(eq? a b)

#f

3) Special Forms

- and
- or
- begin
- cond
- if
- define
- lambda
- quote

There are 2-3 others, but not relevant now/maybe at all?

For functions in Python
1) evaluate operator
2) evaluate operands
3) body

Python
>>> 0 and 1/0
0

3rd way of making lists

```
scm>(and 0 (/ 1 0))
0
scm> (or #t 0 5 3 9)
scm> (begin (+ 5 3) (* 6 2) (/ 6 2))
3
scm> [(if (=2 2) 5 0)
5
```

predicat true case false case

```
if    2==2:
    return 5
else:
    return 0
```

```
if

elif

else
```

```
(if    (= 2 2)    0
            (if (= 3 4)  5  6))
```

```
if    2==2:
    return 0
elif  3==4:
    return 5
```

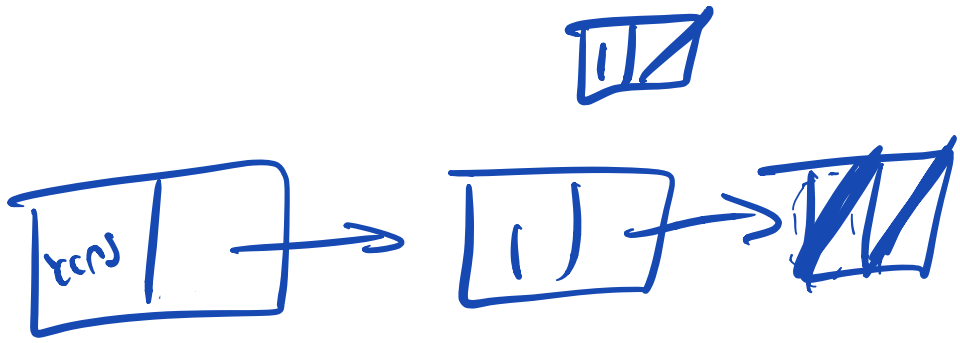
```
else:
    return 6
```

```
(cond ((=22) 0)
      ((=34) 5)
      (else 6))
```

```
(quote 5)
5
```

```
scm>(quote (1 23))
(1 23)
```

```
scm>(quote (cons 1 nil))
```




scm> (quote (1 2 3))

'(1 2 3)

} same

(quote 5)

'5

} same

1)

```
scm> (define pi 3.14)
pi
scm> pi
3.14
scm> 'pi        (quote pi)
pi
scm> (+ 1 2)
3
scm> (+ 1 (* 3 4))
13
scm> (if 2 3 4)
3
scm> (if 0 3 4)
3
```

```
scm> (- 5 (if #f 3 4))
1
```

(quack hello)

```
scm> (if (= 1 1) 'hello 'goodbye)
hello
scm> (define (factorial n) ...
  ...)
factorial
scm> (factorial 5)
120
```